AUTOM-08: Migration Automation

> **Series:** AUTOM — Dynatrace Automation | **Notebook:** 8 of 9 | **Created:** January 2026 | **Last Updated:** 07/30/2026

Configuration migration is the process of transferring Dynatrace settings from one environment to another. This is common in tenant consolidation, Managed-to-SaaS migration, and disaster recovery scenarios.

Table of Contents

Prerequisites

Before starting this notebook, ensure you have:

Requirement	Description
Source tenant access	Admin access to source environment
Target tenant access	Admin access to target environment
API Tokens	Tokens on both source and target
Monaco or Terraform	Migration tool installed

Learning Objectives

By the end of this notebook, you will:

- Understand common migration scenarios
- Know how to export and import configurations
- Be able to handle non-portable configurations
- Validate migration completeness

---

1. Introduction

Common Migration Scenarios

Scenario	Description
Managed to SaaS	Moving from self-hosted to Dynatrace SaaS
Tenant Consolidation	Merging multiple tenants into one
Environment Cloning	Creating dev/staging from production
Disaster Recovery	Restoring config to a new tenant
Config Backup	Periodic export for safekeeping

The 90/10 Rule

> **Important:** 90% of configurations migrate automatically, but the remaining 10% takes 90% of the effort.

Plan extra time for:
- Credentials and secrets
- Custom integrations
- Entity ID references
- Network-specific settings

2. Migration Scenarios

What Can Be Migrated

Configuration Type	Portable	Notes
Management Zones	Yes	Rules migrate, entity IDs don't
Auto-tagging Rules	Yes	Fully portable
Alerting Profiles	Yes	May need zone ID updates
Dashboards	Partial	Entity IDs need updating
SLOs	Yes	Metric expressions portable
Synthetic Monitors	Partial	Location IDs may differ
Request Attributes	Yes	Fully portable
Calculated Services	Yes	Fully portable

What Cannot Be Migrated

Configuration Type	Why Not	Action Required
Credentials Vault	Security isolation	Re-enter manually
API Tokens	Tenant-specific	Generate new tokens
Historic Data	Not transferable	Accept baseline gap
Cloud Integrations	Credential-dependent	Reconfigure
SSL Certificates	Tenant-specific	Re-upload
Private Locations	Infrastructure-specific	Redeploy

Migration Tool Comparison

Tool	Best For	Pros	Cons
Monaco	Config-as-code teams	YAML-based, Git-friendly	Manual entity ID handling
Terraform Export	IaC environments	Full HCL output	Large state files
SaaS Upgrade Assistant	M2S migration	Guided, UI-based	Limited to M2S
Settings API	Custom scripts	Full control	Most manual effort

---

3. Monaco Migration

Step 1: Download from Source

```
# Set source environment
export DT_SOURCE_URL="https://source-tenant.live.dynatrace.com"
export DT_SOURCE_TOKEN="&lt;your-source-api-token&gt;"

# Download all configurations
monaco download \
  --environment-url "$DT_SOURCE_URL" \
  --api-token "$DT_SOURCE_TOKEN" \
  --output-folder ./migration-export
```

Step 2: Review and Clean

After download, review the exported configs:

```
# List exported configuration types
ls -la migration-export/

# Count configurations per type
find migration-export -name "config.yaml" | wc -l
```

Configuration Cleanup

Remove or update:

Item	Action
Entity IDs	Replace with selectors or names
Location IDs	Map to target locations
Credential references	Update to target vault entries
Environment-specific URLs	Update endpoints

Step 3: Create Target Manifest

manifest.yaml:

```
manifestVersion: 1.0

projects:
  - name: migration
    path: migration-export

environmentGroups:
  - name: default
    environments:
      - name: target
        url:
          type: environment
          value: DT_TARGET_URL
        auth:
          token:
            type: environment
            value: DT_TARGET_TOKEN
```

Step 4: Validate and Deploy

```
# Set target environment
export DT_TARGET_URL="https://target-tenant.live.dynatrace.com"
export DT_TARGET_TOKEN="&amp;lt;your-target-api-token&amp;gt;"

# Validate first
monaco validate manifest.yaml

# Dry run
monaco deploy manifest.yaml --environment target --dry-run

# Deploy
monaco deploy manifest.yaml --environment target
```

Handling Entity ID References

Dashboards and alerting profiles often contain entity IDs. These need special handling:

Option 1: Use Entity Selectors

Replace hardcoded IDs with selectors:

```
// Before (hardcoded ID)
{
  "entityId": "HOST-ABC123DEF456"
}

// After (selector)
{
  "entitySelector": "type(HOST),entityName(\"web-server-01\")"
}
```

Option 2: Entity Mapping Script

```
import json
import re

def replace_entity_ids(config: dict, mapping: dict) -&gt; dict:
    """Replace entity IDs using a source-&gt;target mapping."""
    config_str = json.dumps(config)

    for source_id, target_id in mapping.items():
        config_str = config_str.replace(source_id, target_id)

    return json.loads(config_str)

# Build mapping from entity names
entity_mapping = {
    "HOST-ABC123": "HOST-XYZ789",
    "SERVICE-DEF456": "SERVICE-UVW012"
}
```

---

4. Terraform Export

Using the Export Utility

The export utility is **not a separate download** — it is the provider binary itself, invoked with `-export`. After `terraform init` the executable sits under `.terraform/providers/registry.terraform.io/dynatrace-oss/dynatrace/<version>/<os_arch>/`.

```
# Linux / macOS
./terraform-provider-dynatrace -export [-ref] [-migrate] [-import-state] [-id]
[-flat] [-exclude] [&lt;resourcename&gt;[=&lt;id&gt;]]

# Windows
terraform-provider-dynatrace.exe -export [options] [&lt;resourcename&gt;
[=&lt;id&gt;]]
```

Credentials are supplied through **environment variables**, not command-line arguments:

Variable	Required	Purpose
`DYNATRACE_ENV_URL`	Yes	Source tenant endpoint

Variable	Required	Purpose
`DYNATRACE_API_TOKEN`	Yes	API token for the source tenant
`DYNATRACE_TARGET_FOLDER`	No	Output directory (default: `./configuration`)

```
export DYNATRACE_ENV_URL="https://source-tenant.live.dynatrace.com"
export DYNATRACE_API_TOKEN="&lt;your-source-api-token&gt;"
export DYNATRACE_TARGET_FOLDER="./terraform-export"

./terraform-provider-dynatrace -export -ref -id
```

> **Never pass the token as a command-line argument.** Anything on the command line is visible to every user who can run `ps` on that host, and it lands in shell history and CI job logs. The utility reads credentials from the environment for exactly this reason. AUTOM-04 §3 covers the provider auth model in full.

Migrating Between Tenants — the `-migrate` Flag

For tenant-to-tenant moves (Managed to SaaS, or consolidation) use `-migrate` rather than `-ref`. The two are mutually exclusive: `-ref` emits data-source references for a tenant you will keep managing in place, while `-migrate` resolves dependencies for recreating configuration in a *different* tenant.

Approach	Command	Use when
Bulk	`./terraform-provider-dynatrace -export -migrate`	Target tenant is fresh, with no configuration to preserve
Iterative	`./terraform-provider-dynatrace -export -migrate -datasources <resourcename>`	Target already holds configuration you must not overwrite

Export with the environment pointed at the source, then repoint it at the target to apply. The migration guide also documents source-scoped variables (`DYNATRACE_SOURCE_ENV_URL`, `DYNATRACE_SOURCE_API_TOKEN`) so both endpoints can be held at once.

> **Entity IDs do not survive the move.** A migrated OneAgent can register as a *new* entity in the target, which silently breaks any configuration pinned to an entity ID. Re-check everything that references a specific entity after apply — this is the single most common post-migration surprise.

Export Output Structure

Output defaults to a **module structure** — one directory per resource family — written to `DYNATRACE_TARGET_FOLDER`, or `./configuration` if unset. Pass `-flat` to put everything in a single directory instead.

```
configuration/
├── main.tf
├── providers.tf
├── dynatrace_management_zone_v2/
│   └── *.tf
├── dynatrace_autotag_v2/
│   └── *.tf
├── .flawed/              # deprecated configs requiring modification before
apply
└── .required_attention/  # items missing essentials (e.g. credential payloads
the API cannot return)
```

Triage `.flawed/` and `.required_attention/` before committing anything — the second directory is where secrets that the source API refuses to return end up, and they must be re-entered by hand. **Dashboards are excluded by default**; name the resource explicitly to opt in, or run `-list-exclusions` to see the full default-exclusion list.

Importing to the Target

```
cd configuration

# Repoint the provider at the target tenant
export DYNATRACE_ENV_URL="https://target-tenant.apps.dynatrace.com"
export DYNATRACE_API_TOKEN="&amp;lt;your-target-api-token&amp;gt;"

terraform init
terraform plan     # expect a large first plan — review it before applying
terraform apply
```

Adding `-import-state` to the original export runs `terraform init` and imports the exported resources into state automatically, if you want a bootstrapped workspace rather than HCL files alone.

The full `-export` flag reference lives in **AUTOM-04 §8**. The end-to-end Managed-to-SaaS Terraform walkthrough — export, triage, repoint, apply, verify — is in **M2S-95 LAB**, and is not repeated here.

Sources: Terraform export utility (DT docs), Terraform migration guide (DT docs).

---

5. SaaS Upgrade Assistant

For Managed-to-SaaS migrations, Dynatrace provides a guided tool.

Accessing the Assistant

1. Log into your Dynatrace account
2. Navigate to **Apps** → **SaaS Upgrade Assistant**
3. Follow the guided workflow

Assistant Features

Feature	Description
Discovery	Automated inventory of source environment
Compatibility Check	Identify configs that need manual work
Bulk Export	Export all compatible settings
Progress Tracking	Visual dashboard of migration status
Validation	Post-migration validation checks

Upload Format

The SaaS Upgrade Assistant requires configuration archives in **`.tar.gz` format** (not `.zip`). The archive must contain:

```
configurationExport-&lt;datetime&gt;/
├── exportMetadata.json          # Cluster UUID, Monaco version, timestamp
└── export/                      # Monaco download output
    └── saas/&lt;tenantId&gt;/
        ├── &lt;config-type&gt;/
        │   ├── config.yaml
        │   └── *.json
        └── ...
```

Create the archive with:

Platform	Command
Bash (macOS/Linux)	`tar -czf archive.tar.gz configurationExport-<datetime>`
PowerShell (Windows 10+)	`tar -czf archive.tar.gz configurationExport-<datetime>`

> **Note:** Windows 10 version 1803+ and Windows 11 include `tar.exe` natively. On older Windows, use 7-Zip to create the tar.gz.

When to Use

Dynatrace sanctions **three** approaches for Managed-to-SaaS configuration migration. The Assistant is the recommended default, but it is not the only supported path — if you already run config-as-code, use the tool you already run.

Scenario	Recommended Tool	Notes
Managed to SaaS — no config-as-code today	**SaaS Upgrade Assistant**	Recommended default. Guided UI, compatibility check, progress tracking.
Managed to SaaS — already running Monaco	**Monaco**	Download from Managed, deploy to SaaS with a retargeted manifest (§3 above).
Managed to SaaS — already running Terraform	**Terraform** (`-export -migrate`)	Repoint the provider at the SaaS tenant. Full walkthrough in **M2S-95 LAB**.
SaaS to SaaS	Monaco	

Scenario	Recommended Tool	Notes
Backup/Restore	Monaco or Terraform	
GitOps workflow	Monaco or Terraform	

Whichever path you pick, one set of settings **never** migrates automatically and must be recreated by hand: extension and cloud credential configurations (AWS, Azure, GCP, Cloud Foundry, Kubernetes), access tokens and personal access tokens, problem-notification integrations (Jira, OpsGenie, PagerDuty, and the rest), mobile symbolication and JavaScript error settings, request naming and merged services, multi-dimensional analysis saved views, account management (users, groups, permissions), tags and custom entity names, and process-grouping rules — which must be migrated *before* the upgrade, not after. Budget for this explicitly: it is the 10% from §1 that consumes 90% of the effort.

Source: Migrate configuration (DT docs).

---

6. Validation and Verification

Pre-Migration Checklist

Check	Description
[ ] Source inventory	Document all configurations
[ ] Credential list	Identify all secrets to re-enter
[ ] Entity dependencies	Map entity ID references
[ ] Network requirements	Verify target connectivity
[ ] Token scopes	Ensure sufficient permissions

Post-Migration Validation

Run these DQL queries on the target tenant to verify entity counts:

> **Note:** `fetch dt.settings` is NOT a valid DQL data object. Settings objects must be queried via the Settings API (`GET /api/v2/settings/objects`), not DQL. The cells below document the correct approach.

```
// Count management zones
// NOTE: fetch dt.settings is NOT a valid DQL data object.
// Settings objects cannot be queried via DQL.
// Use the Settings API instead:
//   GET /api/v2/settings/objects?schemaIds=&lt;schemaId&gt;
// Example:
//   GET /api/v2/settings/objects?schemaIds=builtin:management-zones
//   GET /api/v2/settings/objects?schemaIds=builtin:tags.auto-tagging
```

```
// Count auto-tagging rules
// NOTE: fetch dt.settings is NOT a valid DQL data object.
// Settings objects cannot be queried via DQL.
// Use the Settings API instead:
//   GET /api/v2/settings/objects?schemaIds=&lt;schemaId&gt;
// Example:
//   GET /api/v2/settings/objects?schemaIds=builtin:management-zones
//   GET /api/v2/settings/objects?schemaIds=builtin:tags.auto-tagging
```

```
// Verify synthetic monitors — Smartscape form (preferred for new queries)
smartscapeNodes "BROWSER_MONITOR"
| summarize total = count()

// Classic form — still functional, and a genuine fallback:
//   fetch dt.entity.synthetic_test
//   | summarize total = count()
//
// dt.entity.synthetic_test maps to the BROWSER_MONITOR Smartscape node type.
Both forms
// returned the same monitor count when live-verified 07/30/2026, so either
works today;
// dt.entity.* is deprecated, so prefer the Smartscape form for anything new.
//
// Two things to know before porting a synthetic query:
//   - Clickpath steps are a SEPARATE node type (BROWSER_MONITOR_STEP), not
fields of the
//     monitor. A query that reads per-step data must traverse to those nodes
rather than
//     expect step attributes on the monitor node.
//   - The other synthetic types map too: dt.entity.http_check -&gt;
HTTP_MONITOR,
//     dt.entity.multiprotocol_monitor -&gt; NETWORK_AVAILABILITY_MONITOR,
//     dt.entity.synthetic_location -&gt; SYNTHETIC_LOCATION.
```

Validation Script

Compare source and target configuration counts:

> **Two things to know before running this.** The SLO schema is `builtin:monitoring.slo` — **not** `builtin:slo`, which does not exist. Querying a nonexistent schema returns `totalCount: 0` rather than an error, so a count-comparison script using the wrong ID reports `0 == 0` and a cheerful match for a domain it never actually checked. This script previously carried that bug.
>
> All four schemas above are also marked **Blocked at upgrade** in AUTOM-02's catalog. After a tenant upgrades to the latest Dynatrace they return zero on both sides, and this script will again report a clean match — for configuration that no longer exists. Count-parity validation is only meaningful while both tenants are on the same generation; across a Classic → Gen3 boundary you need to compare the *replacement* constructs instead.

```
import requests

def count_settings(url: str, token: str, schema_id: str) -&gt; int:
    """Count settings objects of a given schema."""
    response = requests.get(
        f"{url}/api/v2/settings/objects",
        params={"schemaIds": schema_id},
        headers={"Authorization": f"Api-Token {token}"}
    )
    return response.json().get("totalCount", 0)

def validate_migration(source_url, source_token, target_url, target_token):
    """Compare configuration counts between tenants."""
    schemas = [
        "builtin:management-zones",
        "builtin:tags.auto-tagging",
        "builtin:alerting.profile",
        "builtin:monitoring.slo"
    ]

    results = []
    for schema in schemas:
        source_count = count_settings(source_url, source_token, schema)
        target_count = count_settings(target_url, target_token, schema)

        results.append({
            "schema": schema,
            "source": source_count,
            "target": target_count,
            "match": source_count == target_count
        })

    return results
```

Troubleshooting Common Issues

Issue	Cause	Solution
Missing configs	Schema not exported	Export specific schema
Deploy errors	Entity ID invalid	Use selectors instead
Dashboard empty	Data not migrated	Expected (historic data doesn't migrate)

Issue	Cause	Solution
Alerts not firing	Credential issues	Re-enter webhook credentials
Synthetic failing	Location mismatch	Update location IDs

---

7. Summary

Migration Best Practices

Practice	Description
Plan thoroughly	Document everything before starting
Export first	Keep a backup of source config
Validate often	Check counts after each step
Test in dev	Validate in non-production first
Document exceptions	Note what needed manual work

Quick Reference: Migration Commands

Task	Monaco Command
Download all	`monaco download --output-folder ./export`
Download specific	`monaco download --api builtin:management-zones`
Validate	`monaco deploy manifest.yaml --dry-run` — Monaco ships no standalone `validate` subcommand (see AUTOM-03 §5)
Dry run	`monaco deploy manifest.yaml --dry-run`
Deploy	`monaco deploy manifest.yaml`

Series Complete

Congratulations! You've completed the AUTOM series. Here's what you learned:

Notebook	Key Takeaway
AUTOM-01	Choose the right tool for your use case
AUTOM-02	Settings API is the foundation
AUTOM-03	Monaco for GitOps-style management
AUTOM-04	Terraform for state management
AUTOM-05	Workflows for event-driven automation
AUTOM-06	SDKs for custom applications
AUTOM-07	CI/CD for automated deployments
AUTOM-08	Migration patterns and validation

Additional Resources

- Monaco Documentation
- Terraform Provider
- Dynatrace API Reference
- Terraform export utility (DT docs)
- Terraform migration guide (DT docs)
- Migrate configuration — Managed to SaaS (DT docs)
- M2S Migration Series - For Managed-to-SaaS specific guidance, including the **M2S-95 LAB** Terraform migration walkthrough

---

> **Key Takeaway:** Successful migration requires planning, the right tools, and thorough validation. Use Monaco or Terraform for bulk export/import, but always plan for manual work on credentials and entity references.

Next: AUTOM-09: Terraform GitOps Setup Recipe for the operational architecture of standing up a Terraform GitOps shop (repo layout, state backends, lifecycle protections, team onboarding). For Managed-to-SaaS migrations, see the M2S series.
